Name: Menna Hesham Ali – Group: SE2

# Microservices and Cloud-Native Design

## Part B - Implement product-service:

Started the service:

```
PS D:\CMC_labs\lab4\product-service> python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.1.6:5001
Press CTRL+C to quit
127.0.0.1 - - [26/Apr/2026 15:24:19] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [26/Apr/2026 15:24:39] "GET /products/1 HTTP/1.1" 200 -
127.0.0.1 - - [26/Apr/2026 15:24:50] "GET /products/99 HTTP/1.1" 404 -
```

- Valid product:

```
PS C:\Users\PC> curl http://localhost:5001/products/1


StatusCode        : 200
StatusDescription : OK
Content           : {"id":1,"name":"Laptop","price":1200}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 38
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 12:24:39 GMT
                    Server: Werkzeug/3.1.8 Python/3.14.4

                    {"id":1,"name":"Laptop","price":120...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 38], [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 12:24:39 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 38
```

- Invalid product:

```
PS C:\Users\PC> curl http://localhost:5001/products/99
curl : {"error":"Product not found"}
At line:1 char:1
+ curl http://localhost:5001/products/99
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
   eption
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
```

```
PS D:\CMC_labs\lab4\order-service> python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://192.168.1.6:5002
Press CTRL+C to quit
127.0.0.1 - - [26/Apr/2026 15:36:46] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [26/Apr/2026 15:38:59] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [26/Apr/2026 15:40:07] "POST /orders HTTP/1.1" 400 -
127.0.0.1 - - [26/Apr/2026 15:40:51] "POST /orders HTTP/1.1" 201 -
127.0.0.1 - - [26/Apr/2026 15:42:06] "POST /orders HTTP/1.1" 400 -
```

- Valid order:

```
PS C:\Users\PC> Invoke-RestMethod -Method POST `
>>   -Uri "http://localhost:5002/orders" `
>>   -ContentType "application/json" `
>>   -Body '{"product_id": 1, "quantity": 2}'

message       product quantity total_price
-------       ------- -------- -----------
Order created Laptop         2        2400
```

- Invalid order:

```
PS C:\Users\PC> Invoke-RestMethod -Method POST `
>>   -Uri "http://localhost:5002/orders" `
>>   -ContentType "application/json" `
>>   -Body '{"product_id": 99}'
Invoke-RestMethod : {"error":"invalid product"}
At line:1 char:1
+ Invoke-RestMethod -Method POST `
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExc
   eption
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

The order-service communicates with product-service using HTTP requests.

- It sends a request to /products/<id>
- Waits for a response
- Uses that data to create an order

This is an example of **synchronous communication**, where one service depends on another in real time.

## Part D - Containerize both services:

Docker images built successfully for both microservices:

```
PS D:\CMC_labs\lab4> docker build -t product-service ./product-service
[+] Building 13.9s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.1s
 => => transferring dockerfile: 215B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                     1.5s
 => [internal] load .dockerignore                                                                                       0.1s
 => => transferring context: 2B                                                                                         0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e459  0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e459  0.1s
 => [internal] load build context                                                                                       0.1s
 => => transferring context: 767B                                                                                       0.0s
 => CACHED [2/5] WORKDIR /app                                                                                           0.0s
 => [3/5] COPY requirements.txt .                                                                                       0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                            9.7s
 => [5/5] COPY app.py .                                                                                                 0.1s
 => exporting to image                                                                                                  2.0s
 => => exporting layers                                                                                                 1.1s
 => => exporting manifest sha256:807aad36f66fc6e34fd154968ae201709ff5827f35f1b673daef6edfd82e2868                       0.0s
 => => exporting config sha256:2a989b547850b2377c61202396b27a939733c91c83361eb398c5e068f05d0cf3                         0.0s
 => => exporting attestation manifest sha256:a993fb7b72fdd2c29102b5290daa02e8f13a2ec77828c1901724f22ff52948c7           0.1s
 => => exporting manifest list sha256:5e472f79fadf93267e08bc762954e20dba9c6740bc85db5aa6fa21ebbef70edc                 0.0s
 => => naming to docker.io/library/product-service:latest                                                               0.0s
 => => unpacking to docker.io/library/product-service:latest                                                            0.6s
```

```
PS D:\CMC_labs\lab4> docker build -t order-service ./order-service
[+] Building 15.3s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.1s
 => => transferring dockerfile: 215B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                     0.4s
 => [internal] load .dockerignore                                                                                       0.1s
 => => transferring context: 2B                                                                                         0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e459  0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e459  0.1s
 => [internal] load build context                                                                                       0.1s
 => => transferring context: 1.58kB                                                                                     0.0s
 => CACHED [2/5] WORKDIR /app                                                                                           0.0s
 => [3/5] COPY requirements.txt .                                                                                       0.2s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                           11.7s
 => [5/5] COPY app.py .                                                                                                 0.1s
 => exporting to image                                                                                                  2.3s
 => => exporting layers                                                                                                 1.3s
 => => exporting manifest sha256:3686bbb24bba8d87133723059d29f4e5c94c56fbcbd37419baf652cf81874d19                       0.0s
 => => exporting config sha256:7cacd97a02c9715520280b39fe6cc53f82af62ee4b22045b0f261189caa3a32b                         0.0s
 => => exporting attestation manifest sha256:1d0e13e661f911a821715926f46984368302ea6382a847dfdef236c242ccba8e           0.1s
 => => exporting manifest list sha256:aad6219edb2303694e756eaf10aa9aa929763784f927b233f8c05b1bc7ddf3c3                 0.0s
 => => naming to docker.io/library/order-service:latest                                                                 0.0s
 => => unpacking to docker.io/library/order-service:latest                                                              0.7s
```

Both microservices running successfully inside Docker containers:

```
PS C:\Users\PC> curl http://localhost:5001/health


StatusCode        : 200
StatusDescription : OK
Content           : {"service":"product-service","status":"up"}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 44
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 13:08:22 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"service":"product-service","stat...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 44], [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 13:08:22 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 44
```

```
PS C:\Users\PC> curl http://localhost:5002/health


StatusCode        : 200
StatusDescription : OK
Content           : {"service":"order-service","status":"up"}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 42
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 13:08:32 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"service":"order-service","status...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 42], [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 13:08:32 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 42
```

This setup demonstrates:

- Independent deployment units
- Isolation between services
- Reproducible runtime environments

### Part E - Define orchestration with Docker Compose & Part F - Build and run the system:

Docker Compose successfully running microservices:

```
 ✔Image lab4-order-service   Built                                                    2.9s
 ✔Image lab4-product-service Built                                                    2.9s
 ✔Network lab4_default       Created                                                  0.1s
 ✔Container product-service  Started                                                  0.9s
 ✔Container order-service    Started                                                  1.2s
PS D:\CMC_labs\lab4> docker compose ps
NAME              IMAGE                  COMMAND           SERVICE           CREATED          STATUS
PORTS
order-service     lab4-order-service     "python app.py"   order-service     22 seconds ago   Up 21 seconds (healthy)
0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
product-service   lab4-product-service   "python app.py"   product-service   22 seconds ago   Up 21 seconds (healthy)
0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
```

This confirms that:

- containers are running correctly
- services are responding to health checks
- orchestration is functioning properly

Docker Compose acts as a lightweight orchestration tool that simulates cloud-native platforms by managing multi-container applications with networking, health checks, and lifecycle management.

**Part G - Functional testing:**

1. Test product-service:

```
PS D:\CMC_labs\lab4> curl http://localhost:5001/health


StatusCode        : 200
StatusDescription : OK
Content           : {"service":"product-service","status":"up"}
```

```
PS D:\CMC_labs\lab4> curl http://localhost:5001/products/1


StatusCode        : 200
StatusDescription : OK
Content           : {"id":1,"name":"Laptop","price":1200}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 38
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 13:37:25 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"id":1,"name":"Laptop","price":12...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 38],
                    [Content-Type, application/json], [Date, Sun, 26
                    Apr 2026 13:37:25 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 38



PS D:\CMC_labs\lab4> curl http://localhost:5001/products/99
curl : {"error":"Product not found"}
At line:1 char:1
+ curl http://localhost:5001/products/99
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebReq
   uest:HttpWebRequest) [Invoke-WebRequest], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.
   PowerShell.Commands.InvokeWebRequestCommand
```

**Product-service testing:**

- Health endpoint returned service status
- Valid product requests returned correct JSON data
- Invalid product requests returned HTTP 404 error

**2. Test order-service:**

```
PS D:\CMC_labs\lab4> Invoke-RestMethod -Method POST `
>>   -Uri "http://localhost:5002/orders" `
>>   -ContentType "application/json" `
>>   -Body '{"product_id": 1, "quantity": 2}'

message       product quantity total_price
-------       ------- -------- -----------
Order created Laptop         2        2400


PS D:\CMC_labs\lab4> Invoke-RestMethod -Method POST `
>>   -Uri "http://localhost:5002/orders" `
>>   -ContentType "application/json" `
>>   -Body '{"product_id": 99}'
Invoke-RestMethod : {"error":"invalid product"}
At line:1 char:1
+ Invoke-RestMethod -Method POST `
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebReq
   uest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.
   PowerShell.Commands.InvokeRestMethodCommand
```

Order-service testing:

- Successfully created orders by communicating with product-service
- Calculated total price correctly
- Handled invalid product cases gracefully

- **Conclusion**

The system behaves as expected in a fully containerized environment, confirming correct implementation of microservices architecture.

**Part H - Failure injection and resilience observation:**

Stop product-service simulate failure:

```
PS D:\CMC_labs\lab4> docker compose ps
NAME              IMAGE                  COMMAND           SERVICE           CREATED          STATUS
PORTS
order-service     lab4-order-service     "python app.py"   order-service     26 minutes ago   Up 26 minutes (healthy)
0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
product-service   lab4-product-service   "python app.py"   product-service   26 minutes ago   Up 26 minutes (healthy)
0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
PS D:\CMC_labs\lab4> docker stop product-service
product-service
PS D:\CMC_labs\lab4> Invoke-RestMethod -Method POST `
>>   -Uri "http://localhost:5002/orders" `
>>   -ContentType "application/json" `
>>   -Body '{"product_id": 1, "quantity": 2}'
Invoke-RestMethod : {"error":"product-service unavailable"}
At line:1 char:1
+ Invoke-RestMethod -Method POST `
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExc
   eption
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

- **Restart service:**

```
PS C:\Users\PC> docker start product-service
product-service
```

```
PS D:\CMC_labs\lab4> docker compose ps
NAME              IMAGE                  COMMAND           SERVICE           CREATED          STATUS
PORTS
order-service     lab4-order-service     "python app.py"   order-service     29 minutes ago   Up 29 minutes (healthy)
0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
product-service   lab4-product-service   "python app.py"   product-service   29 minutes ago   Up 2 minutes (healthy)
0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
```

Failure Injection and Resilience Testing

To evaluate system resilience, the product-service was intentionally stopped while the system was running under Docker Compose.

Observations:

- The order-service attempted to communicate with product-service
- Requests failed due to service unavailability
- The system returned an error response

**Part I – Architecture Reflection task**

**1. Q: Which parts of this lab show the benefits of microservices over a monolith?**

This lab demonstrates several key advantages of microservices compared to a monolithic architecture.

First, the system is divided into two independent services: product-service and order-service. Each service has a clearly defined responsibility. The product-service handles product data retrieval, while the order-service manages order creation and business logic. This separation of concerns improves modularity and makes the system easier to understand and maintain.

Second, each service can be developed, built, and deployed independently. For example, the product-service can be updated without modifying the order-service, as long as the API contract remains unchanged. In a monolithic system, even a small change would require redeploying the entire application.

Third, containerization with Docker ensures that each service runs in an isolated environment. This improves consistency across development and deployment environments and reduces dependency conflicts.

Finally, the microservices approach enables better scalability. If the order-service experiences higher load, it can be scaled independently without scaling the product-service unnecessarily.

## 2. Q: Which new complexities were introduced by splitting the system into two services?

Although microservices provide flexibility, they also introduce additional complexity compared to a monolithic system.

The first major complexity is network communication. In a monolith, function calls are internal and fast, but in this system, services communicate over HTTP. This introduces network dependency, latency, and potential failure points.

The second complexity is service coordination and discovery. The order-service must know the address of the product-service. In this lab, this is managed using environment variables and Docker Compose service names, but in real systems, service discovery tools would be required.

The third complexity is failure handling. If the product-service is unavailable, the order-service must handle timeouts, retries, and error responses. This requires additional logic that does not exist in a monolithic system.

The fourth complexity is debugging and monitoring. Since the system is distributed, logs and errors are spread across multiple services, making troubleshooting more difficult.

## 3. Q: What would break if network latency increased or one service became slow?

If network latency increases or one service becomes slow, the entire system behavior would be significantly affected due to synchronous communication.

For example, the order-service depends on the product-service to validate product information. If the product-service becomes slow, the order-service will experience delayed responses. This leads to increased response time for the user and may cause request timeouts.

If the latency becomes extreme or the product-service becomes completely unavailable, the order-service will fail to complete requests, even though it is functioning correctly. This creates a cascading failure effect.

Although retry logic and timeouts are implemented in this lab, they only provide basic resilience. In real-world systems, additional patterns such as circuit breakers, caching, or asynchronous messaging would be required to prevent system-wide degradation.

## 4. Q: Which 12-factor app principles are visible in this implementation?

This implementation demonstrates several principles from the 12-factor app methodology:

### 1. Codebase

Each microservice has its own codebase and Docker image, reinforcing the principle of one codebase per service.

**2. Dependencies**
Dependencies are explicitly declared in requirements.txt, ensuring reproducible builds and avoiding hidden system dependencies.

**3. Configuration**
The order-service uses environment variables (e.g., PRODUCT_SERVICE_URL) instead of hardcoded values, following the principle of storing configuration in the environment.

**4. Backing Services**
The product-service acts as a backing service for the order-service. It is treated as an external resource accessed via HTTP.

**5. Port Binding**
Each service is self-contained and exposes its own port (5001 and 5002), meaning each service binds directly to a network port.

**6. Logs**
Logs are generated through container stdout/stderr and can be accessed via Docker logs, following the principle that logs should be treated as event streams.

**7. Disposability**
Containers can be started and stopped quickly using Docker Compose. This supports fast startup and shutdown, which aligns with disposability principles.